具身智能中的运动控制强化学习：完整教程

课程首页 · English · PDF

项目	说明
适用范围	Isaac Lab / RSL-RL 工作流为说明性示例；任务、训练器、资产和执行器版本需一起绑定
文档核验日期	2026-10-18
许可	CC BY 4.0 · Xiong Zeng and contributors

核验边界： 文档整理与上游引用快照核对；未进行教程环境、训练或硬件验证。 版本与固定源码引用见版本说明。原创教材采用 CC BY 4.0，第三方引用、图片及代码遵循其原有许可；注明来源不等于拥有再许可权。

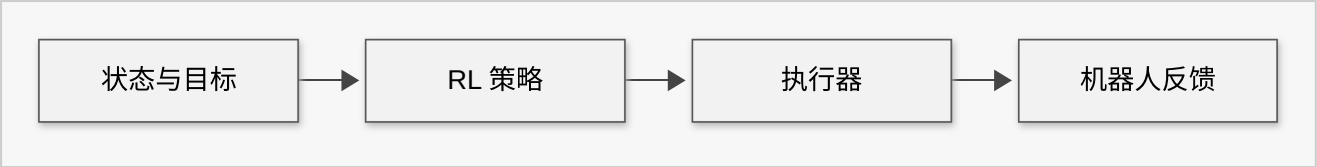
目录

- 第一章：什么是运动控制强化学习?
  - 1.1 运动控制 RL 在具身智能系统中的位置
  - 1.2 运动控制 RL 与具身基础模型 RL 后训练的区别
- 第二章：运动控制强化学习的任务分类
  - 2.1 按机器人任务分类
  - 2.2 按训练方法分类
- 第三章：运动控制强化学习的算法
  - 3.1 PPO：大规模仿真训练的常见基线
  - 3.2 SAC：强调交互数据复用
  - 3.3 PPO 与 SAC 对比
- 第四章：如何设计一个运动控制强化学习任务?
  - 4.1 Observation：机器人能观察到什么?
  - 4.2 Action：强化学习输出什么?
  - 4.3 Reward：机器人应该优化什么?
  - 4.4 Reset：从什么状态开始训练?
- 第五章：训练完整的运动控制策略
  - 5.1 标准训练流程
  - 5.2 第一步：建立机器人仿真模型
  - 5.3 第二步：设置控制频率
  - 5.4 第三步：并行训练
- 第六章：运动模仿强化学习
  - 6.1 基本流程
  - 6.2 Motion Tracking Reward
  - 6.3 DeepMimic
  - 6.4 AMP：Adversarial Motion Priors
  - 6.5 Motion Tracking 与 AMP 的区别

    - 6.6 从单条轨迹到通用运动控制
- 第七章：教师–学生强化学习
    - 7.1 为什么需要教师–学生？
    - 7.2 标准训练过程
    - 7.3 Asymmetric Actor-Critic 与 Teacher–Student 的区别
    - 7.4 RMA：Rapid Motor Adaptation
- 第八章：案例一——使用 PPO 训练 Unitree G1 行走
    - 8.1 任务定义
    - 8.2 Observation
    - 8.3 Action
    - 8.4 Reward
    - 8.5 Curriculum Learning
    - 8.6 开源实验入口
- 第九章：案例二——机械臂与灵巧手运动控制 RL
    - 9.1 任务定义
    - 9.2 建议的控制架构
    - 9.3 三种训练路线
    - 9.4 Observation 设计
    - 9.5 Reward 设计
    - 9.6 训练路线
- 第十章：Sim-to-Real
    - 10.1 为什么仿真成功不保证真机成功？
    - 10.2 Domain Randomization
    - 10.3 Sim-to-Sim
    - 10.4 真机部署检查清单
- 第十一章：如何评估运动控制 RL？
    - 11.1 腿式机器人指标
    - 11.2 Motion Tracking 指标
    - 11.3 灵巧手操作指标
- 第十二章：开源框架与工具
    - 12.1 Isaac Lab
    - 12.2 RSL-RL
    - 12.3 MuJoCo Playground
    - 12.4 如何选择？
- 第十三章：代表性论文与阅读顺序
- 第十四章：学习与研究路线
    - 阶段一：基础运动控制 RL
    - 阶段二：Motion Tracking RL
    - 阶段三：Teacher–Student 与 Sim-to-Real
    - 阶段四：复杂运动控制

    - 阶段五：通用运动控制器
- 第十五章：总结与关键结论
- 参考资料与开源链接

章节知识地图



第一章：什么是运动控制强化学习？

1.1 运动控制 RL 在具身智能系统中的位置

在具身智能中，可以将机器人控制系统粗略分为三个层次：

层次	主要任务	常见技术
高层：任务规划	决定机器人应该做什么	LLM、VLM、VLA、任务规划
中层：运动规划与技能控制	决定机器人如何运动	Motion Planning、Motion Tracking、RL
底层：执行器控制	让电机准确执行指令	PID、PD、阻抗控制、电流控制

运动控制 RL 主要应用于中层，也可能直接学习部分底层控制。例如，机器人的高层控制器要求“以 0.5 m/s 向前走”，运动策略决定关节动作，底层控制器则负责执行。

```
用户指令：“走到桌子前，拿起杯子”
                |
                v
          VLM / VLA / Planner
                |
                v
          目标速度 / 目标位姿
                |
                v
          RL Motion Policy <--- 传感器反馈
                |
                v
         关节目标 / 末端指令
                |
                v
          PD / IK / 电机驱动
                |
                v
             真实机器人
```

一个目标条件运动策略可以写为：

$$a_t = \pi_\theta(o_t, c_t),$$

其中，o_t 是关节、IMU、视觉等观测；c_t 是目标速度、目标姿态或技能指令；a_t 是输出的关节位置、力矩或末端动作；π_θ 是学习得到的策略。

传统控制通常由工程师显式设计控制律，运动控制 RL 则让策略通过与动力学环境的交互学习控制规律。但它们并不互斥：现代系统经常把 RL 与 PD、IK、MPC、WBC（Whole-Body Control）等技术组合。

1.2 运动控制 RL 与具身基础模型 RL 后训练的区别

维度	运动控制 RL	具身基础模型 RL 后训练
主要目标	学习稳定、灵活的运动技能	提高预训练策略的任务成功率
常见输入	关节、IMU、目标状态等	图像、语言、机器人状态等
常见输出	关节控制指令、末端控制指令	Action Chunk、动作分布等
典型模型	MLP、GRU、Transformer	VLA、Diffusion Policy、Flow Matching
常用方法	PPO、SAC、模仿 RL	PPO、SAC、Offline RL、专门的后训练方法
主要交互来源	大量仿真交互	预训练数据、仿真或真机交互
典型任务	行走、跳跃、爬楼梯、灵巧操作	视觉/语言条件下的操作任务
重点难题	稳定性、鲁棒性、运动性能	泛化、样本效率、在线适应

例如，训练 Unitree G1 在速度指令下行走属于运动控制 RL；对已经能抓取的 $\pi_{0.5}$ 通过 RL 提升新环境抓取成功率属于具身基础模型 RL 后训练。两者边界并不绝对：机械臂操作的低层控制 RL 与 VLA 后训练有时优化的是同一机器人动作，只是模型结构、初始化和目标不同。

第二章：运动控制强化学习的任务分类

2.1 按机器人任务分类

类别	英文	典型任务	主要挑战
腿式运动控制	Legged Locomotion	行走、奔跑、转向	平衡、接触稳定性
高动态运动控制	Agile Locomotion	跳跃、爬楼梯、越障、翻滚	动态接触、强非线性
全身动作跟踪	Whole-Body Motion Tracking	跳舞、动作模仿、轨迹跟踪	全身协调、动作保真度
机械臂操作控制	Manipulation Control	抓取、搬运、插入	接触、精度、力控制
灵巧手操作控制	Dexterous Manipulation	手内旋转、重抓取、工具操作	高维动作、多点接触
移动操作与全身协同	Loco-Manipulation	边走边抓、搬运、推门	行走与操作协同

这些任务并不互斥。人形机器人搬箱子通常同时涉及行走、全身协调和操作。

A. Legged Locomotion：腿式运动控制

目标是根据给定指令稳定运动。常见速度指令为：

$$c_t = (v_x^{\text{cmd}}, v_y^{\text{cmd}}, \omega_z^{\text{cmd}}),$$

分别对应前后、横向与转向速度。RL 输出关节动作，以跟踪指定速度。四足和双足机器人已经有较成熟的大规模仿真训练范式。代表工作 *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning* 利用并行仿真训练 ANYmal 运动策略并进行实机验证。[1]

B. Agile Locomotion：高动态运动控制

高动态任务包括跳跃、爬楼梯、跨越障碍、奔跑、跌倒恢复和空中姿态调整。区别于普通行走，策略必须主动利用动力学与接触，例如在爬楼梯过程中协调足部抬升、落脚时序与重心转移。

C. Whole-Body Motion Tracking：全身动作跟踪

给机器人参考动作 $q^{\mathrm{ref}}(t)$，通过 RL 尽量准确地跟踪，同时维持动力学可行性。

```
人类视频 / 动捕 -> 动作重建 -> 机器人重定向
                                  |
                                  v
                            参考机器人动作
                                  |
                                  v
                          RL Motion Tracking
                                  |
                                  v
                            物理可实现的动作
```

代表性路线包括 DeepMimic、AMP、HOVER、BeyondMimic；它们分别涉及参考轨迹模仿、运动先验学习、全身多模式控制和复杂人形动作。[2][3][4]

D. Manipulation Control：机械臂操作控制

目标是完成抓取、搬运、插入等物体操作。RL 可以输出关节动作，也可以输出末端位置或姿态增量，随后交给 IK、操作空间控制（OSC）或关节控制器。通常更关注末端精度、接触力、成功率、碰撞安全与扰动恢复。

E. Dexterous Manipulation：灵巧手运动控制

多指灵巧手需要通过多个关节协同控制接触：

$$a_t = [a_t^{\mathrm{finger1}}, a_t^{\mathrm{finger2}}, \ldots, a_t^{\mathrm{fingerN}}]^\top.$$

任务包括稳定抓取、手内旋转、重抓取、工具使用以及物体打滑后的恢复。核心困难是高维动作空间与复杂的手指–物体接触动力学。

F. Loco-Manipulation：移动操作

这类任务将运动与操作联动，例如边走边搬运、扶墙跨越障碍、行走时双手操作物体和推门。重点从单个技能转向对身体、双臂、腿和接触模式的整体协调。

2.2 按训练方法分类

方法	英文	核心思想	适用场景
从零开始强化学习	RL from Scratch	用任务奖励直接探索	基础平衡、行走

方法	英文	核心思想	适用场景
动作模仿强化学习	Imitation RL	参考运动奖励引导 RL	跳舞、动作复现
残差强化学习	Residual RL	在已有控制器上学习修正	轨迹修复、接触补偿
分层强化学习	Hierarchical RL	高层选技能、低层控制	多阶段任务
教师–学生训练	Teacher–Student	先训练特权教师，再蒸馏	视觉控制、Sim-to-Real
自适应控制策略	Adaptive RL	根据历史观测适应变化	摩擦、负载、动力学变化

这些方法可以组合：先通过模仿强化学习训练教师，再将策略蒸馏到真实传感器可观测的学生模型，最后进行部署。

第三章：运动控制强化学习的算法

3.1 PPO：大规模仿真训练的常见基线

PPO（Proximal Policy Optimization）属于 On-policy 强化学习，在四足、双足、动作跟踪和灵巧操作中使用广泛。典型训练循环：

```
初始化 Actor / Critic
         |
         v
创建数千个并行仿真环境
         |
         v
Actor 交互并收集 (Observation, Action, Reward)
         |
         v
计算 Advantage
         |
         v
更新 Actor 和 Critic -> 重复
```

PPO 特别适合结合 GPU 并行物理仿真：虽然历史交互数据复用有限，但数千个环境可以快速产生大量样本。例如，4096 个环境同时模拟各自的机器人。[1]

常见实现是 RSL-RL，提供面向机器人控制的 PPO、相关网络与训练组件。[5]

3.2 SAC：强调交互数据复用

SAC（Soft Actor-Critic）是 Off-policy 算法，使用 Replay Buffer 反复利用以往交互数据。

```
Robot / Simulator
        |
        v
   Experience
        |
        v
  Replay Buffer
     /     \
    v       v
Critic     Actor
Update     Update
```

```
    \     /
     v   v
    New Policy
```

对于真实机器人，交互成本高，SAC 的数据复用能力有吸引力。它可用于连续机械臂操作、有限真机数据在线改进、灵巧手及残差控制。但在高吞吐 GPU 仿真下，PPO 可能更易实现、调试和扩展；不能脱离环境与交互预算直接断言 SAC 总是更好。

3.3 PPO 与 SAC 对比

特性	PPO	SAC
类型	On-policy	Off-policy
历史样本复用	低	高
Replay Buffer	通常不需要	需要
大规模并行仿真	非常成熟	可行，但系统设计更复杂
高维连续动作	支持	支持
真机样本效率	通常较低	通常有潜在优势
工程实现	相对直接	相对复杂
腿式运动控制	很常见	相对较少
机械臂与灵巧手	常见	也常见/可用
推荐起点	仿真运动控制基线	数据昂贵的连续控制

建议：双足/四足仿真运动控制先复现 PPO；如果关心有限真实数据下的在线策略改进，进一步比较 SAC。算法比较应固定环境、交互次数与评估协议。

第四章：如何设计一个运动控制强化学习任务？

完整环境通常包括：

$$\text{Environment} = \{\text{Observation}, \text{Action}, \text{Reward}, \text{Reset}, \text{Dynamics}\},$$

目标条件任务还需要 Command 或 Goal。实际表现往往比单纯更换 PPO 网络结构更依赖环境建模。

4.1 Observation：机器人能观察到什么？

观测	含义	典型来源
Joint Position	关节位置	编码器
Joint Velocity	关节速度	编码器/估计器
Base Angular Velocity	机身角速度	IMU
Projected Gravity	重力方向在机身坐标系中的表示	姿态估计
Previous Action	上一步控制动作	控制器

观测	含义	典型来源
Velocity Command	目标速度	上层控制
Contact State	足部/手指接触状态	接触估计
Height Map	地形高度	深度相机或其他传感器

腿式机器人的典型观测可以写为：

$$o_t = [q_t, \dot{q}_t, \omega_t, g_t, a_{t-1}, c_t].$$

要区分**仿真可获得的状态**与**真机实际可用的传感器观测**。机身线速度、接触状态和精确物体位姿等在真机上可能需要估计。

Observation History

运动控制往往部分可观测：当前关节角度不能直接确定之前是否滑动、是否受到扰动或执行器是否延迟。可以输入历史窗口：

$$o_t^{\text{hist}} = [o_{t-k}, \dots, o_{t-1}, o_t].$$

随后使用 MLP、GRU 或 Transformer 编码，帮助策略隐式估计隐藏动力学与接触状态。

4.2 Action：强化学习输出什么？

动作空间	输出	特点
Joint Position	关节目标位置	容易构建稳定基线
Joint Velocity	关节目标速度	适合速度控制接口
Joint Torque	关节目标力矩	表达更直接，训练与安全挑战更高
Cartesian Action	末端位置、姿态或速度	适合机械臂

A. Joint Position Control

最常见的一种形式：

$$q_t^{\text{target}} = q^{\text{default}} + s_a a_t, \qquad a_t = \pi_\theta(o_t).$$

由底层 PD 执行：

$$\tau_t = K_p(q_t^{\text{target}} - q_t) - K_d \dot{q}_t.$$

对动作跟踪，也可以设定：

$$q_t^{\text{target}} = q_t^{\text{ref}} + s_a a_t.$$

这对应参考运动上的残差动作。优势是减轻 RL 学习底层电机动态的负担。

B. Joint Torque Control

直接输出 $\tau_t = \pi_\theta(o_t)$，控制自由度更高，但必须显式考虑电流、力矩、温度、速度和关节限位约束。

C. Cartesian Control

机械臂可输出 $a_t = [\Delta x, \Delta y, \Delta z, \Delta R]$，通过 IK 或 OSC 将末端目标转换为关节指令。

```
RL Policy -> 末端增量/目标位姿 -> IK / OSC -> 关节命令 -> 机械臂
```

其中 ΔR 可由旋转向量等表示，实际应保证旋转参数化与控制频率一致。Isaac Lab 提供多种关节和笛卡尔动作接口。[6]

选型建议

机器人	建议初始动作空间
四足/双足	Joint Position Target
人形动作跟踪	Joint Position Target / Residual
常规机械臂	Cartesian Delta Pose
多指灵巧手	Finger Joint Position Target
机械臂 + 灵巧手	Arm Cartesian Action + Finger Joint Action

策略输出与驱动器最终执行的接口不必相同，例如策略输出末端位姿增量，底层电机仍执行关节位置控制。

4.3 Reward：机器人应该优化什么？

常见拆分：

$$r_t = r_t^{\text{task}} + r_t^{\text{stability}} - r_t^{\text{regularization}}.$$

以双足速度跟踪为例：

$$r_{\text{velocity}} = \exp\left(-\frac{\|v_t - v_t^{\text{cmd}}\|^2}{\sigma_v^2}\right),$$

$$r_{\text{orientation}} = \exp\left(-\frac{\|e_t^{\text{orientation}}\|^2}{\sigma_R^2}\right),$$

$$r_{\text{smooth}} = -\|a_t - a_{t-1}\|^2.$$

可组合成：

$$r_t = w_v r_{\text{velocity}} + w_R r_{\text{orientation}} - w_a\|a_t - a_{t-1}\|^2 - w_\tau\|\tau_t\|^2.$$

实际任务还可能加入：身体高度、足部滑动、碰撞、关节限位、能耗、接触稳定等项。只奖励移动速度可能导致身体前冲、异常高速摆腿或利用仿真接触误差；但惩罚过强则可能使机器人选择不运动。

原则：奖励既要反映任务是否完成，也要约束运动是否稳定、自然、可部署。

4.4 Reset：从什么状态开始训练？

普通行走可以从站立开始；动作跟踪和多阶段操作不能总从同一状态开始，否则难以学习中途恢复。

Uniform Reference State Initialization（Uniform RSI） 从参考运动不同时间点采样：

$$t_0 \sim U(0, T).$$

Event-biased RSI 在起跳、落地、接触切换、开始抓取或释放等关键事件附近增加采样密度。初始化应尽量满足动力学和接触一致性，避免把不合理的跳跃状态强加给策略。

第五章：训练完整的运动控制策略

5.1 标准训练流程

```
机器人模型 (URDF / MJCF / USD)
            |
            v
物理仿真器 (Isaac Lab / MuJoCo)
            |
            v
定义 Observation / Action / Reward / Reset
            |
            v
      PPO / SAC 训练
            |
            v
       仿真性能测试
            |
            v
 Domain Randomization / Curriculum
            |
            v
      Sim-to-Sim 验证
            |
            v
       Sim-to-Real
            |
            v
         真实机器人
```

5.2 第一步：建立机器人仿真模型

模块	应建模的内容
Robot Model	连杆质量、惯量、关节轴、关节限位
Actuator	电机力矩、速度限制、摩擦与响应
Contact	摩擦、碰撞几何、接触参数
Sensors	编码器、IMU、视觉传感器
Control	PD 增益、控制频率、动作延迟
Environment	地面、障碍物、操作物体

URDF 不仅是可视化模型，也是重要的动力学参数载体；但“文件里有参数”不代表“参数经过真机标定”。

5.3 第二步：设置控制频率

以下仅为示例，并非某一具体机器人的真实配置：

模块	示例频率
Physics Simulation	500 Hz
Joint PD Control	500 Hz
RL Policy	50 Hz
Vision Update	30 Hz
High-level Planning	5 Hz

若 $f_{\text{sim}} = 500\,\text{Hz}$、$f_{\text{RL}} = 50\,\text{Hz}$，每次 RL 输出后进行 10 个物理步长，即 Action Decimation。高频控制器负责执行，RL 在较低频率更新目标。

5.4 第三步：并行训练

一个**示例性** PPO 训练配置：

参数	示例配置
Parallel Environments	4096
RL Frequency	50 Hz
Rollout Length	24 steps/environment
Algorithm	PPO
Actor / Critic	MLP / MLP
Action	Joint Position Target
Observation	Proprioception + Command

一次 Rollout 的环境交互数为 $4096 \times 24 = 98{,}304$。这一数值是收集到的步数，不代表参数更新次数；视觉或高接触灵巧操作任务往往需要减少并行环境数。

Actor-Critic 结构：

```
Observation
   |
   +-------> Actor  -----> Action -----> Simulator
   |                                        |
   +-------> Critic -----> Value <------- Reward
```

训练期间更新 Actor 与 Critic，典型部署时只运行 Actor。

第六章：运动模仿强化学习

复杂跳跃、跳舞、体操动作和高维灵巧操作，直接从零探索可能非常困难，因此可以引入参考运动。

6.1 基本流程

```
Human Motion / Teleoperation
            |
            v
    Motion Reconstruction
            |
            v
     Motion Retargeting
            |
            v
    Robot Reference Motion
            |
            v
     Motion Tracking RL
            |
            v
      Robust Motion Policy
```

参考数据可以来自动捕、人体视频重建、遥操作、已有控制器以及运动规划。

6.2 Motion Tracking Reward

给定参考关节位置 q_t^{ref}，可以定义：

$$r_{\text{track}} = \exp\left(-\frac{\|q_t - q_t^{\text{ref}}\|^2}{\sigma_q^2}\right).$$

全身任务还可能跟踪 Root Position、Root Orientation、Joint Velocity、End-effector Position、Contact Schedule。仅关节角接近并不保证机器人不会摔倒；需要同时考虑动作相似性、动力学可行性与稳定性。

6.3 DeepMimic

DeepMimic 使用参考运动奖励与任务奖励，帮助物理角色学习复杂动作，是 Motion Tracking RL 的经典工作。[2]

```
Reference Motion -> Tracking Reward --+
                                      +--> RL Policy -> Physics
Task Specification -> Task Reward ----+
```

项目主页：https://xbpeng.github.io/projects/DeepMimic/

6.4 AMP：Adversarial Motion Priors

传统 Motion Tracking 要求在给定时刻跟踪指定轨迹；AMP 使用判别器从参考动作数据中学习运动风格奖励，使策略在完成任务目标的同时产生符合先验的动作。例如，学习自然的行走风格，而不严格逐帧复制某一条轨迹。[3]

6.5 Motion Tracking 与 AMP 的区别

特性	Motion Tracking RL	AMP
是否需要运动数据	通常需要参考轨迹	需要运动数据
是否显式跟踪某条轨迹	是	不一定
奖励来源	跟踪误差	学到的运动先验奖励
主要目标	准确复现参考运动	产生符合参考分布的运动
典型难点	时间同步、跟踪稳定性	判别器训练、奖励平衡

6.6 从单条轨迹到通用运动控制

HOVER 等工作探索将不同控制命令和动作模式集成到统一的全身控制策略中，使策略不再局限于固定轨迹的逐帧跟踪。[4]

趋势：Single Motion Tracking Policy → Multi-Mode / Universal Motion Controller。

第七章：教师–学生强化学习

7.1 为什么需要教师–学生？

在仿真中可以访问精确物体位置/速度、真实摩擦系数、接触力以及隐藏动力学参数，但这些信息在真机中未必可测。如果 Actor 直接依赖它们，就难以部署。

```
仿真完整状态：物体位姿、接触力、摩擦、动力学
                        |
                        v
                  Teacher Policy
                        |
                        v
                  Teacher Action
                        |
                  Distillation
                        |
                        v
真机观测: Camera / Encoder / IMU -> Student Policy
```

7.2 标准训练过程

先训练使用特权状态的教师：

$$a_t^T = \pi_T(s_t).$$

再训练只依赖可部署观测的学生：

$$a_t^S = \pi_S(o_t),$$

典型的动作蒸馏损失：

$$\mathcal{L}_{\text{distill}} = \mathbb{E}\left[\|a_t^S - a_t^T\|^2\right].$$

学生可借助观测历史和状态估计模块推断隐藏信息。实际蒸馏还需处理教师/学生访问状态分布不同的问题。

7.3 Asymmetric Actor-Critic 与 Teacher–Student 的区别

Asymmetric Actor-Critic： Actor 训练时就只使用可部署观测，而 Critic 可使用 Privileged Information。

Teacher–Student： Teacher Actor 可以使用 Privileged Information，再将教师策略迁移给仅使用有限观测的 Student Actor。

所以前者是**训练结构**，后者是**策略迁移方法**。两者可以组合。Isaac Lab 与 RSL-RL 支持相应的观测分组和训练机制。[5][6]

7.4 RMA：Rapid Motor Adaptation

RMA 将动力学适应问题与机器人运动策略结合，利用近期观测历史构建隐式自适应信息，使机器人应对地形、摩擦、负载及未知动力学变化。[7]

论文：https://arxiv.org/abs/2107.04034

第八章：案例一——使用 PPO 训练 Unitree G1 行走

8.1 任务定义

让 G1 根据目标速度 $c_t = [v_x^{\text{cmd}}, v_y^{\text{cmd}}, \omega_z^{\text{cmd}}]$ 稳定运动，包括前后、横向移动与转向。

8.2 Observation

建议先使用本体感知：Joint Position、Joint Velocity、Base Angular Velocity、Projected Gravity、Previous Action、Velocity Command。训练楼梯或复杂地形后，再逐步添加 Height Scan、接触估计、观测历史与视觉特征。

8.3 Action

推荐位置目标：

$$q_t^{\text{target}} = q^{\text{default}} + s_a \pi_\theta(o_t),$$

底层由 PD 执行，利于复用现有控制接口。

8.4 Reward

奖励项	目标
Linear Velocity Tracking	跟踪前进/横向速度
Angular Velocity Tracking	跟踪转向速度
Orientation Stability	维持合理身体姿态
Action Smoothness	减少动作震荡
Joint Limit Penalty	避免极限关节位置

奖励项	目标
Foot Slip Penalty	降低足底滑动
Termination Penalty	惩罚摔倒或失败

8.5 Curriculum Learning

阶段	任务	目的
1	站立	平衡
2	低速前进	基本步态
3	前后左右移动	扩大速度范围
4	转向	角速度跟踪
5	外力扰动	稳定与恢复
6	不平地形	接触适应
7	楼梯	复杂地形运动

可以连续扩展命令/地形分布，并不一定每阶段重训单独策略。

8.6 开源实验入口

Unitree RL Lab 基于 Isaac Lab，提供 Unitree 机器人的训练及部署相关代码。[8] 以下命令为该仓库接口的**示例**，执行前应核对当前版本的环境与任务名称：

```
# 查看可用任务
./unitree_rl_lab.sh -l

# 训练 G1 速度跟踪策略（示例任务名）
./unitree_rl_lab.sh -t --task Unitree-G1-29dof-Velocity

# 测试策略
./unitree_rl_lab.sh -p --task Unitree-G1-29dof-Velocity
```

第一个研究实验：先复现平地行走，再分别修改 Action Smoothness、Domain Randomization、Observation History 和 Foot Slip Penalty，观察训练收敛、速度误差和扰动恢复是否变化。

第九章：案例二——机械臂与灵巧手运动控制 RL

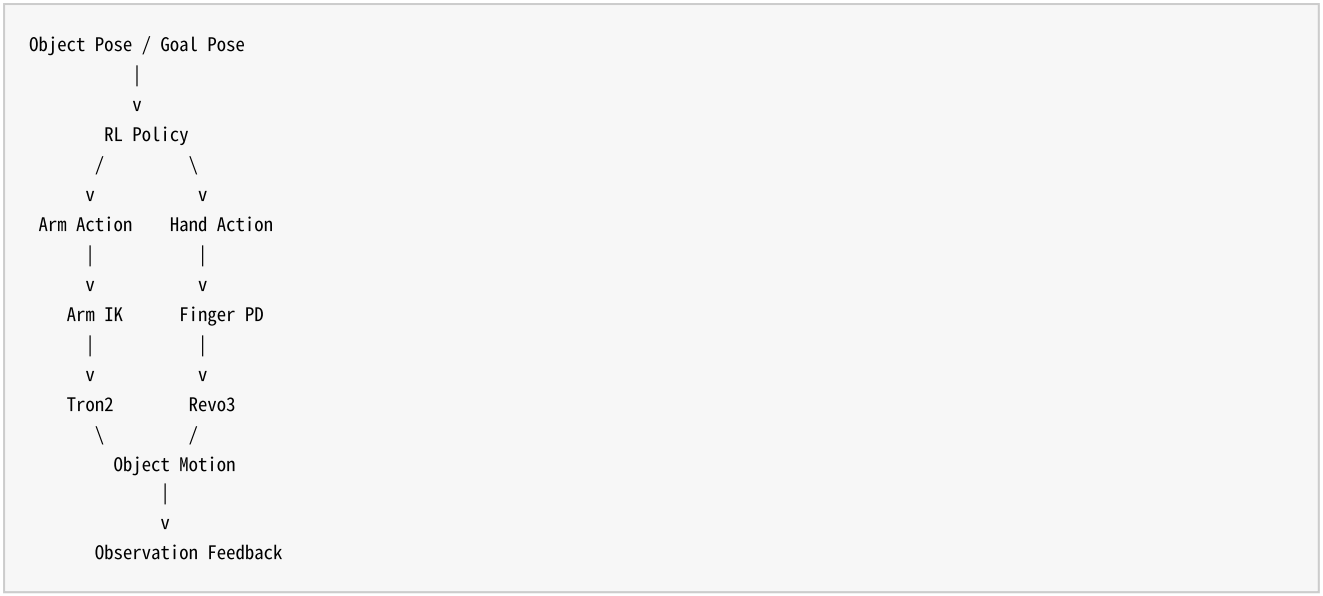
以 **Tron2 机械臂 + Revo3 灵巧手 + 螺丝刀操作** 为例，说明如何把腿式运动控制的环境设计方法迁移到复杂接触操作。

9.1 任务定义

1. 从桌面不同位置拾取螺丝刀。
2. 协调机械臂姿态和手指接触。
3. 移动螺丝刀到指定位置。

4. 完成指定角度旋转。
5. 在扰动后保持或恢复抓取。

9.2 建议的控制架构



可以先训练机械臂与手分离的控制器，再逐步扩展到联合策略。

9.3 三种训练路线

路线 A：从零开始训练。直接使用抓取、抬起、旋转和成功奖励：

$$r_t = r_{\text{grasp}} + r_{\text{lift}} + r_{\text{rotate}} + r_{\text{success}}.$$

优点是不依赖参考轨迹，困难在于高维接触探索。

路线 B：Motion Tracking RL。从人类视频或遥操作获得参考：$T^{\text{ref}}_{\text{hand}}(t)$、$q^{\text{ref}}_{\text{finger}}(t)$。可以降低探索难度，但单条轨迹不足以覆盖所有物体初始姿态和接触变化。

路线 C：Residual RL。在轨迹跟踪器、IK 或已有策略之上学习受约束的动作修正：

$$a_t = a_t^{\text{base}} + \Delta a_t^{\text{RL}}.$$

例如由基线控制机械臂末端跟踪，RL 修正手指接触，或对臂手都施加有限残差。实际必须约束残差幅值、动作变化率和接触力。

9.4 Observation 设计

$$o_t = [q_{\text{arm}}, q_{\text{hand}}, \dot{q}, \hat{T}_{\text{object}}, T_{\text{goal}}, a_{t-1}].$$

其中 $\hat{T}_{\text{object}}$ 由头部/腕部相机的位姿估计器提供。如果真机无法准确测量物体速度，可用历史观测/状态估计器推断。**Actor 不应无意中使用只在仿真中有的真实物体状态**；Critic 可以采用特权状态。

9.5 Reward 设计

奖励	作用
Reach	靠近目标物体
Grasp	形成稳定抓取
Lift	抬起物体
Position	移动到目标区域
Rotation	完成目标旋转
Stability	减少滑动和掉落
Smoothness	抑制动作震荡
Success	奖励完成完整任务

如果只有最终旋转成功奖励，从随机探索中发现合理抓取可能非常困难。建议使用课程学习与适度分阶段奖励。

9.6 训练路线

阶段	任务	目标
1	已抓住时维持稳定	基本接触控制
2	已抓住时旋转	手内操作
3	接近目标并抓取	抓取能力
4	抓取后旋转	多阶段技能
5	物体初始位姿随机化	泛化
6	加物体位姿估计噪声	视觉鲁棒性
7	加外力扰动	恢复能力
8	真机闭环测试	验证迁移

对于有参考轨迹的任务，可将 Uniform RSI 与 Event-biased RSI 结合，使训练更多覆盖接触建立、释放和物体转动等关键片段。

第十章：Sim-to-Real

10.1 为什么仿真成功不保证真机成功?

差异类型	典型来源
Mass / Inertia	质量与惯量误差
Friction	地面或物体摩擦不准确
Actuator	电机响应、死区、内部控制

差异类型	典型来源
Delay	通信/控制延迟
Observation Noise	编码器、IMU、视觉误差
Contact	接触求解器误差
Geometry	碰撞几何不准确
Calibration	关节零位、坐标系和外参误差

例如，仿真中电机对目标动作快速响应，而真实硬件有延迟，就可能导致振荡。

10.2 Domain Randomization

训练时随机化环境/动力学参数：

$$\xi \sim p(\xi),$$

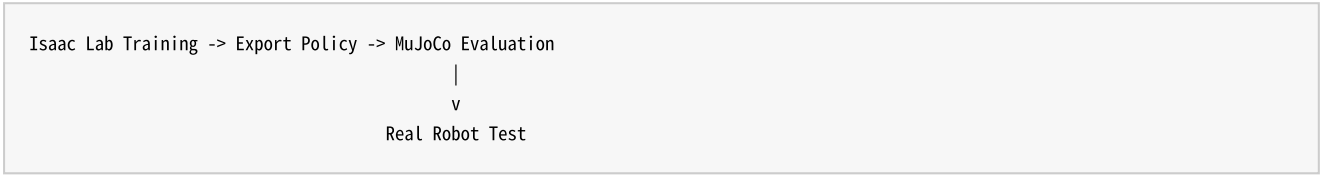
其中 ξ 可以包含质量、重心、摩擦、执行器强度、延迟和传感器噪声。

随机化	目的
Mass	适应质量误差
Center of Mass	适应重心偏差
Friction	适应接触表面变化
Motor Strength	适应执行器差异
Action Delay	适应控制延迟
Observation Noise	适应传感器误差
External Push	提高扰动鲁棒性

随机化范围要依据标定数据、硬件规格和物理常识设置，并不是越大越好。

10.3 Sim-to-Sim

可在 Isaac Lab 中训练，再在 MuJoCo 中测试（或反过来），验证策略是否利用了某个仿真器特有的数值行为。



如果同一控制策略在不同仿真器表现差异很大，应优先检查动力学、接触与动作接口。通过 Sim-to-Sim 并不等于实机安全。

10.4 真机部署检查清单

- 观测维度、顺序和标准化参数与训练一致。
- 关节映射、正负方向、单位、坐标系与零位一致。

- Action Scale、PD 增益、控制频率与仿真设定一致或经过验证。
- 位置、速度、力矩、温度设置保护约束。
- 通信中断、推理超时具有安全回退与紧急停止机制。
- 双足机器人配置防跌倒措施；灵巧手注意异常接触力、卡死和物体掉落。
- 初次部署从低风险、低幅值动作开始，逐步扩展任务范围。

进阶：学习真实与仿真之间的动力学差异

ASAP 等工作尝试利用真实机器人数据对齐仿真与真实动力学，改善敏捷全身动作的 Sim-to-Real。[9]

第十一章：如何评估运动控制 RL？

训练 Reward 上升不代表真实任务表现一定改善。必须构建独立测试集和性能指标。

11.1 腿式机器人指标

指标	含义
Success Rate	任务完成率
Fall Rate	摔倒率
Velocity Tracking Error	速度跟踪误差
Orientation Error	姿态误差
Energy Consumption	能耗
Foot Slip	足底滑动
Recovery Rate	扰动恢复率
Generalization	新地形/新参数下的表现

11.2 Motion Tracking 指标

指标	含义
Joint Tracking Error	关节轨迹误差
Root Position Error	根部位置误差
Root Orientation Error	根部方向误差
End-effector Error	末端误差
Contact Consistency	接触一致性
Motion Completion Rate	动作完成率
Recovery Rate	扰动后的恢复率

11.3 灵巧手操作指标

指标	含义
Task Success Rate	任务成功率
Object Pose Error	物体位姿误差
Grasp Stability	抓取稳定性
Object Drop Rate	物体掉落率
Contact Slip	滑动情况
Recovery Rate	扰动恢复率
Completion Time	任务完成时间
Generalization	新物体位姿下的表现

公平对比原则：相同环境与任务、相同交互预算、多个随机种子、未见过的测试状态，以及与训练奖励独立的性能指标。尤其要区分训练表现、仿真泛化和真机表现。

第十二章：开源框架与工具

框架	主要用途	建议
Isaac Lab	通用机器人仿真与 RL	优先学习
RSL-RL	PPO、运动控制和蒸馏	优先学习
Unitree RL Lab	Unitree 机器人训练与部署	G1/H1 适合
MuJoCo Playground	GPU 加速机器人学习	值得比较
legged_gym	经典腿式机器人 PPO 代码	理解经典实现
skrl	多种 RL 算法实现	算法对比可用

12.1 Isaac Lab

项目：https://github.com/isaac-sim/IsaacLab

支持 Legged Locomotion、Humanoid Locomotion、Manipulation、Dexterous Manipulation 和 Contact-rich Manipulation 等任务。代表性任务名（以特定版本官方环境目录为参考）：[6]

示例环境	任务
`Isaac-Velocity-Flat-G1-v0`	G1 平地速度跟踪
`Isaac-Velocity-Rough-G1-v0`	G1 复杂地形速度跟踪
`Isaac-Velocity-Flat-Unitree-Go2-v0`	Go2 平地行走
`Isaac-Reach-Franka-v0`	Franka 到达
`Isaac-Lift-Cube-Franka-v0`	Franka 举起方块

示例环境	任务
`Isaac-Repose-Cube-Allegro-v0`	Allegro 灵巧手物体重定向

任务名称可能随软件版本变化，以本地 task registry 为准。

12.2 RSL-RL

项目：https://github.com/leggedrobotics/rsl_rl

建议按 Runner → PPO → Actor/Critic → Rollout Storage → Distillation 的顺序读代码。若研究 PPO 运动控制的算法改进，可以直接从这套实现构建基线。[5]

12.3 MuJoCo Playground

项目：https://github.com/google-deepmind/mujoco_playground

提供面向四足、人形、灵巧手、机械臂的 GPU 加速强化学习环境与任务。某些批量动力学任务比完整渲染系统更轻量；实际速度取决于 GPU、接触数量、视觉与仿真后端。[10]

12.4 如何选择？

研究目标	推荐起点
学习 PPO 运动控制	Isaac Lab + RSL-RL
训练 G1 行走	Unitree RL Lab
GPU 大规模并行训练	Isaac Lab / MuJoCo Playground
灵巧手操作	Isaac Lab / MuJoCo Playground
视觉与运动控制结合	Isaac Lab
修改 PPO 算法	RSL-RL
比较 SAC 与 PPO	支持二者的通用 RL 实现
轻量级控制实验	MuJoCo Playground

第十三章：代表性论文与阅读顺序

论文	年份	核心主题	重点学习
DeepMimic	2018	动作模仿 RL	参考运动奖励、RSI
AMP	2021	对抗运动先验	从运动数据学习风格
RMA	2021	快速自适应	历史观测与动力学适应
Learning to Walk in Minutes	2022	并行 PPO	大规模运动训练
Learning Robust Perceptive Locomotion	2022	感知运动控制	地形感知与迁移
HOVER	2025	全身多模式控制	通用控制命令

论文	年份	核心主题	重点学习
ASAP	2025	动力学差异对齐	Sim-to-Real
BeyondMimic	2025	人形复杂动作	动作跟踪与技能泛化
MuJoCo Playground	2025	GPU 机器人学习	现代实验平台

建议优先精读四项：

1. **Learning to Walk in Minutes**：并行 PPO、奖励、课程学习与 Sim-to-Real。[1]
2. **DeepMimic**：参考动作、动作模仿奖励、RSI、物理动作跟踪。[2]
3. **RMA**：特权状态、历史观测、自适应以及部署。[7]
4. **HOVER**：全身控制、多命令模式、动作跟踪与蒸馏。[4]

第十四章：学习与研究路线

阶段一：基础运动控制 RL

目标：训练一个能够稳定行走的双足/四足机器人。

学习 Isaac Lab、RSL-RL、PPO、Observation/Action/Reward、Domain Randomization。先复现 G1 等平台的速度跟踪，再做奖励和观测消融。

阶段二：Motion Tracking RL

目标：跟踪参考动作，同时具备物理可行性与恢复能力。

学习 Motion Retargeting、Tracking Reward、RSI、Motion Curriculum、Residual RL。从单条参考动作开始，再逐步增加初始状态扰动和外部扰动。

阶段三：Teacher–Student 与 Sim-to-Real

目标：策略不依赖仿真中特有的完整状态。

学习 Asymmetric Actor-Critic、Privileged Information、蒸馏、Observation History 与 Domain Randomization。训练特权教师，将其迁移给可部署学生，并开展跨仿真器测试。

阶段四：复杂运动控制

路线	典型任务
Humanoid Locomotion	爬楼梯、越障、跳跃、跌倒恢复
Dexterous Manipulation	手内旋转、工具操作、扰动恢复

深入研究接触动力学、任务课程、强鲁棒性和真机闭环策略。

阶段五：通用运动控制器

探索一个支持多种命令的条件策略：

$$a_t = \pi_\theta(o_t, c_t).$$

其中 c_t 可编码目标速度、目标位姿、参考运动片段、目标物体运动或技能条件。对应方向包括 Goal-conditioned Motion Control、Universal Motion Tracking、Hierarchical Motion Policies、Residual Motion Adaptation 和 Learning-based Whole-Body Control。

第十五章：总结与关键结论

运动控制强化学习可以从**任务、训练方法和工程流程**三个维度理解。

任务	核心问题
Locomotion	如何稳定移动?
Agile Locomotion	如何完成高动态动作?
Motion Tracking	如何准确跟踪参考动作?
Manipulation	如何完成物体操作?
Dexterous Manipulation	如何协调多指接触?
Loco-Manipulation	如何协调整体移动与操作?

方法	重点解决的问题
PPO	如何利用大规模并行仿真学习控制?
SAC	如何增加交互数据的利用效率?
Imitation RL	如何利用参考动作减少探索难度?
Residual RL	如何在已有控制器之上改进?
Teacher–Student	如何从特权仿真状态迁移到真实观测?
Adaptive RL	如何适应未知动力学和环境变化?

完整开发流程为:

$$\boxed{\begin{array}{c}\text{Robot Modeling} \rightarrow \text{Task Design} \rightarrow \text{PPO/SAC Training} \\ \text{Curriculum} + \text{DR} \rightarrow \text{Sim-to-Sim} \rightarrow \text{Sim-to-Real}\end{array}}$$

结论一：任务建模与算法同样重要。 Observation、Action、Reward、Reset、仿真动力学和课程学习会显著影响性能。

结论二：PPO + 并行仿真是重要基线。 腿式运动与动作跟踪尤其成熟；真机数据宝贵时，应进一步考虑 SAC 等高数据复用方法。

结论三：通用、条件化、鲁棒的运动控制器是值得关注的方向。 一条有价值的研究路线是:

Motion Tracking RL → Residual RL → Robust Sim-to-Real → Goal-conditioned Universal Motion Control。

参考资料与开源链接

以下资料主要为教程中提及的论文主页、论文页面和官方开源仓库。论文年份与任务名称应以其最终发表版本/仓库当前版本为准。

1. Rudin et al., *Learning to Walk in Minutes Using Massively Parallel Deep Reinforcement Learning* (2022)
2. Peng et al., *DeepMimic* (2018)
3. Peng et al., *AMP: Adversarial Motion Priors* (2021)
4. HOVER: Versatile Neural Whole-Body Controller for Humanoid Robots；论文
5. RSL-RL 官方仓库
6. Isaac Lab 官方仓库；环境目录
7. RMA: Rapid Motor Adaptation for Legged Robots
8. Unitree RL Lab 官方仓库
9. ASAP: Aligning Simulation and Real-World Physics
10. MuJoCo Playground 官方仓库
11. legged_gym 官方仓库
12. skrl 官方仓库

文档版本：2026-10-15。示例参数用于教学，不能直接视为真实机器人安全部署配置。